

Master IASD – Intelligence Artificielle, Systèmes, Données
Data Science Lab – Project A1

Graph Autoencoders for Sparse Rating Matrices

Does side information help IMC-GAE against a biased-ALS baseline in a very sparse regime?

Paper studied

Inductive Matrix Completion Using Graph Autoencoder
W. Shen, C. Zhang, Y. Tian, L. Zeng, X. He, W. Dou, X. Xu
CIKM 2021

Group 4

Raphaël Breaud · Son Hai Nguyen · Aaron Aidoudi

Supervised by Alexandre Vérine

October 2026

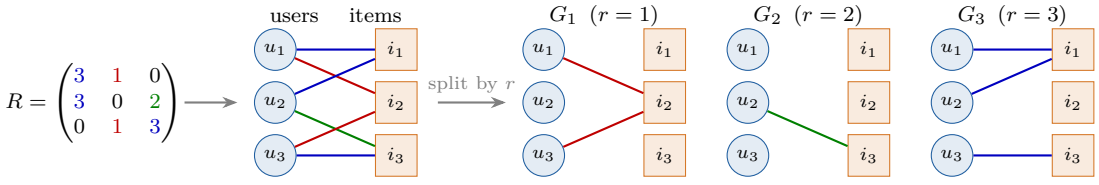
1 Introduction

A recommender system must predict how a user would rate an unseen movie. The ratings form a user–movie matrix with most entries missing (about 1% observed here), and *matrix completion* fills in the rest. Matrix factorisation, e.g. alternating least squares (ALS), predicts a rating from a user and a movie latent vector. Graph methods instead treat ratings as edges of a bipartite graph: a *graph autoencoder* (GAE) encodes each node from its neighbours and decodes a rating from a user and a movie vector. IMC-GAE [1] combines the personal node vectors of earlier GAEs with input vectors shared by all nodes, so that part of each representation depends only on the graph structure.

Our contribution. We re-implement IMC-GAE on a MovieLens dataset and add movie content (genres, year, titles) to this content-free model, asking whether it helps rarely rated movies; the test set is used neither for tuning nor for early stopping.

2 The IMC-GAE model

Rating graphs. Let $R \in \{0, \dots, T\}^{n \times m}$ be the rating matrix ($r_{u,i} = 0$ if unobserved, n number of user and m number of items), Ω the observed pairs and $\mathcal{V}$ the $N = n + m$ nodes. Ratings are split into one bipartite graph per level, $G_t = (\mathcal{V}, \mathcal{E}_t)$ with $\mathcal{E}_t = \{(u, i) \in \Omega : r_{u,i} = t\}$, and $\mathcal{N}_t(v)$ is the neighbourhood of v in G_t (Figure 1).



(a) rating matrix (b) bipartite graph: one edge per observed rating (c) T rating graphs, each containing all $N = n + m$ nodes

Figure 1: Construction of the rating graphs. Observed entries of R become user–item edges (0 = missing, no edge; colour = rating level); the edges are then split by level into $T = 3$ bipartite graphs G_1, G_2, G_3, each containing all users (circles) and items (squares).

Input embeddings. In every G_t, node v starts from $x_t^0[v] = [\, a \,\|\, \rho_{\text{role}(v)} \,\|\, \ell_t(v) \,]$. The *identical* vector a, shared by all nodes, lets message passing aggregate one-hop statistics; the *role-aware* vector $\rho \in \{\rho_{\text{user}}, \rho_{\text{item}}\}$ separates user-to-item from item-to-user messages. Both are *structural* and transfer to unseen nodes. $\ell_t(v)$ is a *personalised* node- and level-specific embedding, replaced by the training-node average for an unseen node.

Encoder. In training, each node is dropped at layer l with probability $p_l = p_0 + l\,\theta$ (p_0 at the first layer, θ per extra layer): $\tilde{x}_t^l[v] = m_t^l[v]\, x_t^l[v]/(1 - p_l)$, $m_t^l[v] \sim \text{Ber}(1 - p_l)$, with the same nodes dropped in all T graphs. Dropping more in deeper layers regularises and keeps a pair's representation local, like IGMC without subgraphs. The L layers are parameter-free normalised convolutions, combined with decreasing weights:

$$x_t^{l+1}[v] = \text{ReLU}\Big(\sum_{j \in \mathcal{N}_t(v)} \frac{\tilde{x}_t^l[j]}{\sqrt{|\mathcal{N}_t(v)|\,|\mathcal{N}_t(j)|}} \Big),\ h_t[v] = \sum_{l=0}^{L} \frac{x_t^l[v]}{l+1},\ h[v] = \sum_{t=1}^{T} h_t[v],\ n[v] = \tanh\big(W h[v]\big). \tag{1}$$

The symmetric normalisation weights each message by both degrees: $1/\sqrt{|\mathcal{N}_t(j)|}$ favours neighbours with few ratings, whose ratings are more specific (as in inverse document frequency, a movie rated by everyone says little about a user's taste), while $1/\sqrt{|\mathcal{N}_t(v)|}$ keeps heavily rated nodes from accumulating large vectors. This weighting is what the shared vectors a and ρ learn from: being identical for all nodes (of a role), they differ after propagation only through the degrees. After one layer, the shared block of v in G_t is $c_t(v)\, a$ with $c_t(v) = \sum_{j \in \mathcal{N}_t(v)} 1/\sqrt{|\mathcal{N}_t(v)|\,|\mathcal{N}_t(j)|}$, so across the T graphs it encodes how often, and with which neighbours, a node gives or receives each rating, like a user's or a movie's mean rating. After encoding, each user and movie is summarised by one vector $n[v]$ combining, over the T rating graphs, its position and local structure in each G_t (who it is linked to, with which rating) with its personal and content information.

Decoder and loss. Each level t gets a *bilinear* score, turned into probabilities by a softmax; the prediction is the expected rating. The loss adds to the cross-entropy a cosine-similarity term that keeps a node's representations at adjacent rating levels close, since ratings are ordered:

$$e_t(u,i) = n[u]^\top W_t\, n[i] = \sum_{k,l} n[u]_k\, (W_t)_{kl}\, n[i]_l, \qquad p_t(u,i) = \frac{\exp e_t(u,i)}{\sum_{s=1}^{T} \exp e_s(u,i)}, \tag{2}$$

$$\hat{r}_{u,i} = \sum_{t=1}^{T} t\, p_t(u,i), \qquad \mathcal{L} = -\frac{1}{|\Omega|} \sum_{(u,i) \in \Omega} \log p_{r_{u,i}}(u,i) \;-\; \lambda \sum_{t=1}^{T-1} \sum_{v \in \mathcal{V}} \cos\big(h_t[v], h_{t+1}[v]\big). \tag{3}$$

Each learned matrix $W_t \in \mathbb{R}^{d \times d}$ defines how well user and movie vectors match for rating t.

3 Our implementation

Data. We use $T = 10$ half-star levels; the graphs contain only the 31,598 training ratings. The 4,980 movies are grouped by training ratings into *few* (< 5; 3,228), *medium* (5–19; 1,404) and *many* (≥ 20; 348).

Deviations from the paper. (i) The output layer has a bias, $n[v] = \tanh(Wh[v] + b)$. (ii) The prediction uses rating values, $\hat{r}_{u,i} = \sum_t c_t\, p_t(u,i)$ with $c_t \in \{0.5, \ldots, 5\}$; it gave a lower in- and out-of-sample RMSE than the most probable level $c_{\arg\max_t p_t}$.

Content features. Each feature set k is a fixed matrix $F_k \in \mathbb{R}^{N \times n_k}$ (zero rows for users) with its own learnable projection $P_k \in \mathbb{R}^{n_k \times d_k}$:

$$x_t^0[i] = \big[\, a \,\|\, \rho_{\text{item}} \,\|\, F_{\text{gy}}[i]\, P_{\text{gy}} \,\|\, F_\tau[i]\, P_\tau \,\|\, \ell_t(i) \,\big], \qquad x_t^0[u] = \big[\, a \,\|\, \rho_{\text{user}} \,\|\, \mathbf{0} \,\|\, \mathbf{0} \,\|\, \ell_t(u) \,\big]. \tag{4}$$

Genre + year: $F_{\text{gy}}[i] = [\, g(i)/|g(i)| \,\|\, y(i) \,] \in \mathbb{R}^{G+1}$, the multi-hot genres divided by their number and the standardised release year (parsed from the title, median if missing). *Title:* "The movie <title>" (year removed) is encoded by the frozen sentence encoder `all-MiniLM-L6-v2` into $\mathbb{R}^{384}$, reduced by PCA to 16 components scaled so rows have norm ≈ 1; only $P_\tau \in \mathbb{R}^{16 \times d_\tau}$ is learned, mapping titles into the GNN input space. Since content enters before message passing, rarely rated movies start from an informative vector that also reaches their raters. We compare no features, genre + year, and genre + year + title.

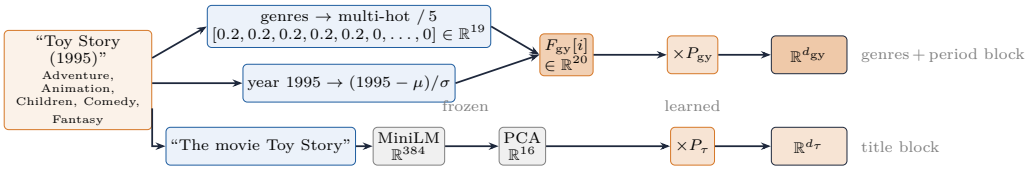


Figure 2: Content encoding of one movie; only P_{gy} and P_τ are learned. The two outputs are the content blocks of $x_t^0[i]$ (Figure 3).

Learnable parameters. Figure 3 summarises the model. Message passing has no weights; with $d_{\text{in}} = d_a + d_\rho + d_{\text{gy}} + d_\tau + d_\ell$ and $d = \dim n[v]$, $|\Theta| = d_a + 2d_\rho + (G{+}1)d_{\text{gy}} + 16\,d_\tau + NTd_\ell + d\,(d_{\text{in}}{+}1) + Td^2$, dominated by the personalised embeddings, i.e. 1.96M parameters for 31.6k ratings, hence weight decay, dropout and early stopping.

	a	$\rho_{\text{user}}, \rho_{\text{item}}$	P_{gy} / P_τ (ours)	$\ell_t(v)$	W, b	$W_1, \ldots, W_T$
shape	d_a	$2 \times d_\rho$	$(G{+}1) \times d_{\text{gy}}$ / $16 \times d_\tau$	$T \times N \times d_\ell$	$d \times d_{\text{in}}$, d	$T \times d \times d$

Code. Data are handled with NumPy; the model is the PyTorch class `IMCGAE`, which chains `NodeEmbedding` (builds the inputs (4)), `Encoder` (dropout, propagation and aggregation, giving $n[v]$) and `Decoder` (bilinear logits, softmax and $\hat{r}_{u,i}$). The optional classes `GenreTerms` (scores from a user's genre-rating profile) and `SumTerms` (sum of such terms) add to the logits.

Training protocol. The class `Trainer` trains the model with full-batch Adam and weight decay, and its method `Trainer.cross_validate` scores each Optuna trial (40 trials) by the mean RMSE of a 3-fold CV (≈21k fit / 10.5k validation), with early stopping (check every 5 epochs, patience 8, best weights restored). The best configuration is refitted on all training ratings for 55 epochs (median best epoch) and saved; the test set is used once (95% bootstrap interval).

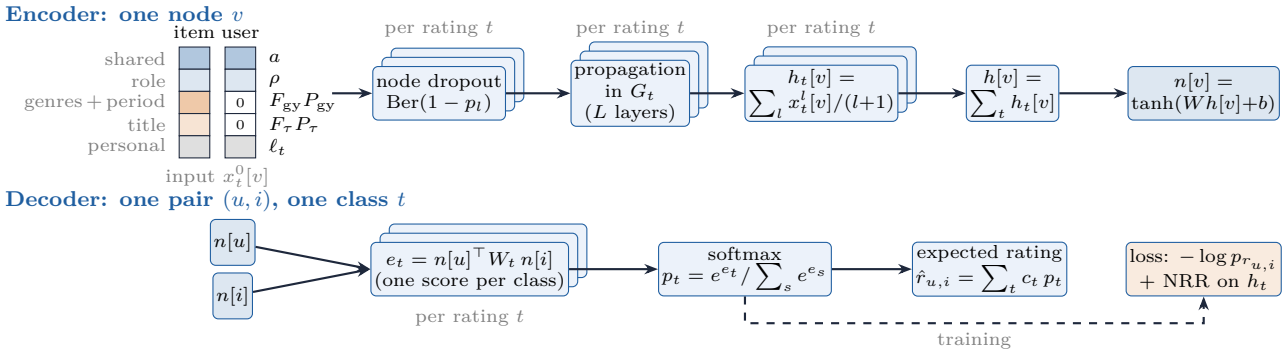


Figure 3: Path of a node vector (encoder) and of a rating class t (decoder); stacked boxes repeat for every t.

4 Experimental setup

We study rating prediction for **610 users** and **4,980 movies**, using **31,598 training ratings** and **31,598 disjoint test ratings** on a $610 \times 4{,}980$ matrix. Ratings lie between 0.5 and 5 in steps of 0.5. All hyperparameter selection uses training data only. Our evaluation criterion is root mean squared error (RMSE),

$$\mathrm{RMSE}(\Omega) = \left(\frac{1}{|\Omega|} \sum_{(u,i)\in\Omega} (\widehat{r}_{ui} - r_{ui})^2 \right)^{1/2}.$$

We consider (i) a *base IMC–GAE* using only the observed rating graph; (ii) *base + year + genre*, adding a standardized release year and 19 normalized genre indicators, expected to help little-rated movies; (iii) *base + year + genre + title*, additionally using 384-dimensional title embeddings compressed to 16 principal components, which may capture semantic similarities. A *bias-aware alternating least squares (ALS)* factorization, combining user/item biases and 20-dimensional latent factors, provides a conventional benchmark.

5 Training and hyper-parameter tuning

Remark. All graphs and results generated by Optuna presented here correspond to Model (ii), which includes the *base + year + genre*. This model achieved the best results compared with the other models. The experimental protocol and hyperparameter search procedure were kept exactly the same for all the models to ensure a fair and consistent comparison.

Optimisation. Training is full-batch with Adam and L2 weight decay: one epoch is one gradient step on all training ratings. In a preliminary check documented in the code, AdamW memorised the training set (validation RMSE ≈ 0.92 against ≈ 0.89), so we kept Adam. The cosine term of the loss is averaged over transitions and nodes instead of summed, so λ is not on the scale of the paper.

Validation protocol. The search objective is a 3-fold cross-validation over rating entries: a seeded random permutation of the 31,598 training ratings is cut into three parts; in each fold the graphs are rebuilt from the other two parts (≈21k ratings) and the held-out part (≈10.5k) is used for validation, which mimics the hidden entries of the completion task. A rating that would leave a user or movie without any training rating stays in the training part, so cold start is never evaluated. Validation RMSE is computed every 5 epochs; a training stops after 8 checks (40 epochs) without an improvement above 10^{-5} (cap: 600 epochs) and the best weights are restored.

Hyper-parameter search. Optuna's TPE sampler (seed 0) runs 40 trials, each scored by the mean fold RMSE at the best epoch of each fold (Table 1), i.e. 120 trainings in 44 min on a laptop CPU (about 66 s per trial).

Hyper-parameter	Range	Best	Hyper-parameter	Range	Best
d_{shared}	$\{8, 16, 32\}$	16	dropout p_0	$[0.05, 0.6]$	0.55
d_{role}	$\{8, 16, 32\}$	8	increment θ	$[0, 0.15]$	0.139
d_{personal}	$\{8, 16, 32, 64\}$	32	learning rate (log)	$[10^{-3}, 2{\cdot}10^{-2}]$	0.0020
d_{feat}	$\{4, 8, 16\}$	4	weight decay (log)	$[10^{-5}, 10^{-1}]$	0.0074
decoder size	$\{32, 64, 128\}$	64	cosine weight λ	$\{0, 0.01, 0.1\}$	0
layers L	$\{1, \ldots, 4\}$	3			

Table 1: Optuna search space and best trial. Here the d_{feat} represent at the same time d_{gy} and d_{τ} where given the model (iii) b*base + year + genre + title* we have $d_{feat} = d_{gy} = d_{\tau}$.

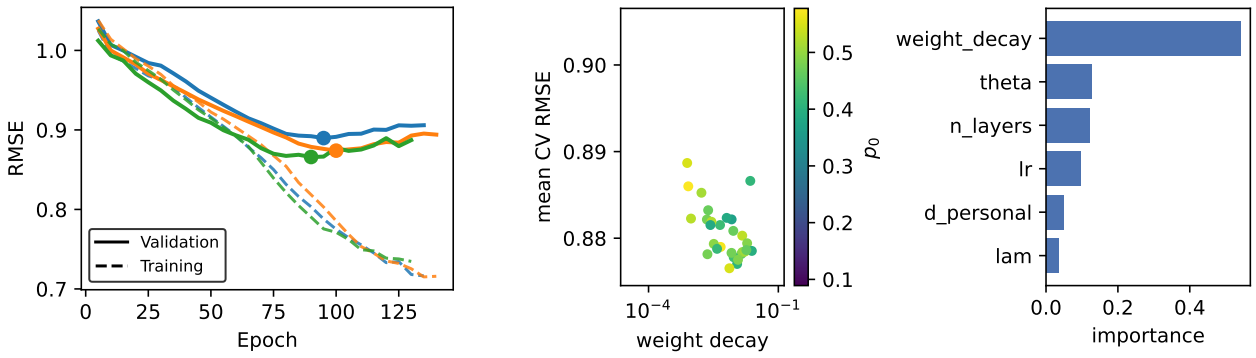


Figure 4: Left: training (dashed) and validation (solid) RMSE in each CV fold of the best configuration; dots mark the best epoch. Right: weight decay against CV RMSE over the 40 trials (colour: p_0), and Optuna importances.

What the search shows.
Observations. The best trial reaches a mean CV RMSE of 0.8765 (folds 0.8897 / 0.8739 / 0.8659); replaying it on the same folds reproduces these three values exactly. At the best epoch the training RMSE is 0.786–0.790, i.e. 0.08–0.10 below the validation RMSE. The five best trials lie within 0.0017 (0.8765–0.8782) and all use three propagation layers and strong node dropout ($p_0 \in [0.40, 0.55]$); four of them set $\lambda = 0$, and the best trial

uses plain cross-entropy. With $\theta = 0.139$, the best trial drops 55%, 69% and 83% of the nodes at the three layers. The learning rate and the number of epochs trade off: the five best trials stop between 40 and 195 epochs for nearly identical CV scores. Optuna's importance (normalised to sum to 1) is largest for weight decay (0.54), then θ (0.13), the number of layers (0.12) and the learning rate (0.10); every other hyper-parameter is below 0.05 (d_{personal} 0.048, λ 0.036, p_0 0.007). The starting vector has dimension 60 (16 shared, 8 role, 4 genre/year, 32 personal) and the output vector 64. The model has 1.96M parameters, 97.5% of them the per-node, per-level embeddings, i.e. 58 parameters per training rating. After training, these embeddings have a typical magnitude of 0.021, five times below their initialisation (0.1), while the shared, role and genre/year parameters stay around 0.09–0.10.

Interpretation. Strong weight decay, strong and depth-increasing dropout and a flat optimum suggest that the search mainly tunes regularisation: with 58 parameters per rating the model is over-parameterised, and the penalty acts mostly on the per-node vectors, whose data gradient is small. That ALS, with 20 factors, does at least as well on test is consistent with this, but we did not test much smaller GNNs. Dropping the deepest layers hardest may limit leakage of the ratings being predicted into their own representation, but this is a hypothesis we did not isolate.

Limits. The best p_0 (0.55) and θ (0.139) lie close to the upper bounds of their ranges (0.6 and 0.15), so the ranges may have been too narrow.

Final model, reproducibility and cost. The best configuration is refit on all training ratings for 95 epochs, the median of the folds' best epochs (95, 100, 90), since no validation set is left; the refit takes about 16 s. It reaches a training RMSE of 0.820 and a test RMSE of 0.8682, against 1.0369 for the training mean; the test set is never used for tuning or early stopping. The checkpoint stores weights, hyper-parameters, feature preprocessing and metrics. Seeds are fixed (fold split, Optuna sampler, PyTorch and NumPy generators, offset by the fold index). Across five refits with different seeds, the test RMSE is 0.8682 ± 0.0023, so differences below about 0.005 between single models are within seed noise. At submission time, `generate.py` loads the saved checkpoint with the lowest recorded test RMSE, refits it on the table it receives and fills the missing cells; on the course platform it obtains RMSE **0.848**.

6 Train, validation and test performance

Table 2 contains the IMC–GAE base model, the IMC–GAE with metadata, the Optuna-tuned full-feature IMC–GAE, and the ALS notebook. The models were evaluated on the same test set, but their cross-validation procedures were different, so the CV results are not directly comparable. The IMC–GAE Base + year + genre model achieves the lowest test RMSE (0.8682), slightly outperforming ALS (0.8720). However, ALS still outperforms the base GAE (0.8893) and the full-feature GAE (0.8764). Despite achieving the lowest training RMSE (0.6879), ALS does not achieve the best test performance, suggesting that a lower training error does not necessarily lead to better generalization. Nevertheless, these results alone are insufficient to draw definitive conclusions about overfitting.

One potential advantage of IMC–GAE is its ability to incorporate movie metadata, such as genre, release year, and title, which may be particularly useful for movies with few observed ratings. Unlike ALS, which relies solely on the rating matrix, GAE can exploit similarities between movies through their features, potentially transferring information from frequently rated movies to less popular ones. The improvement obtained by incorporating year and genre suggests that metadata can enhance predictive performance, although adding title information does not lead to further improvements. To investigate whether metadata is particularly beneficial for less popular movies, we compare the prediction performance of the different models according to the number of training ratings available for each movie.

Table 2: RMSE of saved model checkpoints, the separately Optuna-tuned full-feature GAE, and tuned ALS.

Model	Train	Test
Base GAE	0.8157	0.8893
Base + year + genre	0.7979	**0.8682**
Base + year + genre + title	0.8285	0.8764
Bias-aware ALS *(CV-tuned)*	0.6879	0.8720

To test sensitivity to sparsity, we divide movies by the number of their *training* ratings: fewer than five (3,228 movies; 6,543 test ratings), 5–19 (1,404; 13,107), and at least 20 (348; 11,948). Table 3 uses saved predictions from the three earlier GAE variants and the CV-tuned ALS model.

Table 3: Test RMSE by movie popularity (training-rating count).

Model	Few (< 5)	Medium (5–19)	Many (≥ 20)
Base IMC–GAE	0.9374	0.8884	0.8628
Base + year + genre	**0.89833**	**0.86508**	0.85483
Base + year + genre + title	0.90837	0.87695	0.85790
ALS (CV tuned)	0.9203	0.8693	**0.8473**

Year and genre reduce test error relative to the base model in *all three* popularity groups. Adding title features worsens the corresponding sparse-movie error. We observe that the models incorporating metadata alongside the baseline features outperform ALS. This suggests that metadata may help movies with few ratings to benefit from information shared with other movies that have similar metadata. Where effect goes down as the number of ratings goes up.

7 Representation ablation

For the *Optuna-tuned full-feature GAE only*, we zeroed each component at inference, without retraining. The test RMSE changes are:

Table 4: Inference-time ablation; original tuned-GAE test RMSE = 0.8764.

Removed	Shared	Personal	Role	Year + genre	Title
Test RMSE	1.0982	0.9760	0.9108	0.9160	0.8774
Increase	+0.2218	+0.0996	+0.0344	+0.0396	+0.0010

The **shared graph representation** is indispensable to the fitted predictor, consistent with its encoding of the observed rating graph and neighborhood structure. The **title block has almost no effect**, reinforcing the controlled validation evidence that these title embeddings offer little useful information and may encourage overfitting. Inference-time removal is not equivalent to retraining without a feature; comparable ablations of the other GAE checkpoints were *not available*.

8 Conclusion.

In this project, we evaluated IMC–GAE for movie rating prediction using different metadata configurations and compared its performance with a bias-aware ALS baseline.

The GAE model incorporating release year and genre achieves the lowest test RMSE (0.8682), slightly outperforming ALS (0.8720). However, adding title information does not improve performance, and ALS still outperforms the other GAE variants. These results suggest that metadata can improve prediction accuracy, but additional model complexity does not necessarily lead to better results.

Further experiments on movies with few observed ratings would help determine whether GAE offers a more substantial advantage in sparse-data and cold-start settings.

References

[1] W. Shen, C. Zhang, Y. Tian, L. Zeng, X. He, W. Dou and X. Xu. Inductive matrix completion using graph autoencoder. In *Proc. 30th ACM International Conference on Information and Knowledge Management (CIKM)*, 2021. arXiv:2108.11124.